Année scolaire : 2025-2026
UAA9 : Projet et cahier des charges
Titulaire du cours : Lucas Embrechts
5e Technique de Transition – Informatique

UAA9

Projet et cahier des charges

Nom / Prénom : .....

Classe : .....

		Pondération	Pondération finale
Q1	Implémentation du projet	/100	/45
Q2	Dossier de conception	/100	/25
Q3	Défense Orale	/100	/30
TOTAL /100			

1. Description générale

Les élèves ont pour objectif de réaliser un programme en C (le « code source ») répondant à un besoin d'un client (le « projet »).

Ce projet sera clôturé par une défense orale individuelle.

En plus de l'implémentation du programme, les élèves devront produire un dossier d'analyse et de conception (le « cahier des charges »).

2. Planification du projet sur l'année et délivrables

Le projet débute la première semaine de cours après les congés d'hiver et se déroulera jusqu'à la fin de l'année scolaire.

Plusieurs remises intermédiaires du cahier des charges et du code source rythmeront la période de développement.

La planification complète est disponible dans un document dédié sur Classroom.

3. Cahier des charges

Le cahier des charges sert à documenter et structurer les aspects techniques, fonctionnels et organisationnels du projet, afin de guider son développement et d'assurer sa compréhension et sa reproductibilité. **Il s'agit d'un document essentiel au même titre que le code source.**

Ce dossier reprendra :

1. La définition du projet
2. L'analyse et la conception du projet
3. L'implémentation
4. Une rétrospective
5. Les perspectives

De plus, la première page du dossier comportera une page de garde reprenant :

- le logo de l'école
- l'année scolaire
- les noms et prénoms des membres du groupe
- une image illustrative

3.1. Définition du projet

Cette section reprend la définition complète du projet. En d'autres termes, la description des besoins, les délivrables ainsi que les technologies utilisées.

3.2. Analyse et conception

3.2.1. Interfaces et cas d'utilisation

Cette section contient les interfaces pour les différents cas d'utilisation de votre programme.

Dans un premier temps, dressez une liste complète des cas d'utilisation de votre programme en précisant :

- les acteurs concernés
- les cas d'erreur

Ensuite, pour chaque cas identifié, proposez une interface (ou des exemples de dialogue) de votre programme.

3.2.2. Description des données

Cette section contient une description complète et détaillée des données utilisées et produites par le programme.

Elle doit mentionner, pour chaque source de données :

- son nom (ex. animaux.txt)
- une description générale des données qu'elle contient
- son type : fichier texte, base de données, etc.
- la structure des données qu'elle contient :
  - le type de chaque donnée (ex. un entier)
  - la signification de chaque donnée (ex. l'année de naissance d'un animal)

Vous pouvez également proposer un schéma entités-associations !

3.2.3. Diagramme(s) d'action

Cette section contient les diagrammes d'action (écris en pseudo-code) nécessaires à la réalisation de votre programme.

Vous devez obligatoirement en créer au moins deux :

1. celui de votre fonction principale (main).
2. celui d'une fonction auxiliaire essentielle au bon fonctionnement du programme

Outil conseillé : https://section-ig.github.io/da/

3.3. Implémentation

Cette section vise à documenter le code source.

3.3.1. Structures et énumérations

Décrivez l'ensemble des structures et des énumérations utilisées dans votre programme.

Vous devez également faire le lien avec la partie « Description des données » !

3.3.2. Prototypes

Décrivez l'ensemble des prototypes de vos fonctions auxiliaires. Pour chaque prototype, décrivez les entrées et sorties de la fonction ainsi que ses effets.

Par exemple :

```c
/*
Entrée(s) :
- le nom de l'animal

Sortie :
- la fiche de l'animal

Effets :
crée la fiche d'un animal sur base du nom reçu en entrée ;
le nombre de soins reçus est mis à 0 par défaut
*/
Animal creerFiche(char nom[]);
```

3.3.3. Fonctions

Cette section contient la description de l'ensemble des fonctions du programme (fonction principale `main` et fonctions auxiliaires).

L'objectif (en plus de documenter le code source) est de fournir des notes claires qui permettent à l'élève de relire son travail et de préparer la défense orale, en expliquant l'implémentation de manière détaillée.

Pour chaque fonction, l'élève rédigera une fiche respectant le format suivant :

Signature de la fonction :	*Rappeler le prototype de la fonction (cf. prototype)*
Entrées :	*Rappeler les entrées de la fonction (cf. prototype)*
Sortie :	*Rappeler la sortie de la fonction (cf. prototype)*
Effets :	*Rappeler les effets de la fonction (cf. prototype)*
Contexte d'appel :	*Le(s) prototype(s) de(s) la/les fonction(s) d'où cette fonction est appelée.*
Etapes de l'algorithme :	*3 à 6 puces décrivant le déroulement logique en langage naturel (pas de code).*
Cas d'erreur / limites :	*Ce qui peut se produire (saisie invalide, fichier introuvable, index hors bornes…) et comment la fonction réagit.*
Données manipulées :	*Structures, variables importantes, fichiers concernés (lien avec « Description des données » et « Structures et énumérations »).*
Tests rapides :	*1 à 2 scénarios simples (entrées → résultat attendu).*

Par exemple :

Signature de la fonction :	`Animal creerFiche(char nom[]);`
Entrées :	Le nom de l'animal
Sortie :	La fiche de l'animal
Effet :	Créer et initialiser une fiche Animal à partir du nom fourni, avec les valeurs par défaut nécessaires (dont `nbSoins = 0`).
Contexte d'appel :	Cette fonction est appelée dans la fonction `void ajouterAnimal(void);`
Etapes de l'algorithme :	1. Créer une variable de type `Animal` 2. Copier le nom reçu dans le champ `nom` de la fiche

<table>
<tr><td></td><td colspan="2">3. Initialiser les champs par défaut (<code>nbSoins = 0</code>, <code>actif = true</code>).
4. Retourner la fiche Animal ainsi créée.</td></tr>
<tr><td>Cas d'erreur / limites :</td><td colspan="2">• Si le nom est vide : retourne une fiche avec un nom vide
• Si le fichier des animaux n'existe pas ou n'est pas disponible : retourne une fiche avec un nom vide
• Si le quota d'animaux est atteint : retourne la fiche complétée avec <code>actif = false</code></td></tr>
<tr><td>Données manipulées :</td><td colspan="2">Structure(s) : <code>Animal</code>
Fichier(s) : <code>animaux.dat</code></td></tr>
<tr><td rowspan="3">Tests rapides :</td><td>Entrée(s)</td><td>Résultat</td></tr>
<tr><td>« Jack »</td><td>Crée et retourne la fiche.</td></tr>
<tr><td>« »</td><td>Crée et retourne la fiche avec un nom vide.</td></tr>
</table>

3.4. Rétrospective

Dans cette section, vous présenterez une rétrospective de votre projet : ses forces, ses faiblesses, etc. Si certaines fonctionnalités n'ont pas pu être implémentées, expliquez les raisons.

3.5. Perspectives

Dans cette section, vous présenterez les améliorations de votre programme : que peut-on améliorer ? que peut-on imaginer en plus ? Etc.

4. Code source

- Le programme doit être rédigé en langage C
- Aucune librairie supplémentaire non vue au cours ne peut être utilisée sauf accord des superviseurs.
- L'entièreté du code doit être développé par les membres de l'équipe. Chaque fichier doit présenter un commentaire en en-tête avec le nom de la personne ayant codé ce fichier (si cette partie a été codée en duo, les noms des deux, auteurs doivent être indiqués).
- Les parties plus complexes du code doivent être commentées de manière à permettre à un utilisateur lambda de comprendre et de reprendre facilement le code. C'est-à-dire que pour chaque partie de code répondant à une tâche spécifique, le développeur doit décrire en commentaire ce que le code réalise. Tout commentaire inutile sera sanctionné dans la cotation. Le commentaire doit être précis, pertinent et apporter un éclairage utile sur la logique de la tâche.
- L'ensemble du code source doit être hébergé dans un dépôt GIT sur la plateforme GitHub ; en tout temps, ce dépôt doit être accessible au professeur.

5. Système d'evaluation

5.1. Evaluation du travail journalier

Tout au long de l'année, plusieurs remises intermédiaires seront organisées et constitueront l'évaluation du TJ :

1. Cahier des charges : interfaces et cas d'utilisation
2. Cahier des charges : description des données
3. Cahier des charges : diagrammes d'action
4. Cahier des charges : structures et énumérations
5. Cahier des charges : prototypes
6. Cahier des charges : fonctions
7. Cahier des charges complet (version intermédiaire)

Attention, le TJ ne se limite pas aux remises : l'implémentation du projet pendant les séances seront aussi évaluées.

5.2. Evaluation sommative

La remise finale du projet constitue l'évaluation de l'UAA9.

Le projet est évalué sur base :

- d'une défense orale (30%)
- du cahier des charges (25%)
- du code source (45%)

Évaluation du cahier des charges

Le cahier des charges doit être complet. La partie 1 (*Définition du projet*) est déjà écrite et ne doit pas être modifiée. Il vous est demandé de compléter les parties 2 (*Analyse et Conception*), 3 (*Implémentation*), 4 (*Rétrospective*) et 5 (*Perspectives*). La mise en page, l'orthographe et la qualité de la rédaction seront prises en compte dans l'évaluation.

Évaluation du programme

Le programme sera évalué selon les points suivants :

- le programme s'exécute correctement au niveau des fonctionnalités attendues,
- le code source est de qualité.

Des bonus seront attribués pour :

- l'allocation dynamique,

- ou des fonctionnalités supplémentaires non mentionnées dans le cahier des charges.

Toutefois, ces bonus ne seront attribués que si la note du travail de base atteint les 80%.

Évaluation orale

L'évaluation orale consiste à présenter le projet oralement devant les superviseurs. La présentation se déroulera comme suit :

1. Présentation du logiciel
2. Présentation du code source
3. Questions / Réponses sur le logiciel, le code source et le cahier des charges

La présentation du logiciel ainsi que la présentation du code source ne devra pas dépasser 10 minutes.

Dans le cas où le projet est réalisé par deux élèves, ces derniers doivent impérativement démontrer leur participation au projet. À cette fin, le code source devra être documenté de telle manière que l'on puisse identifier clairement l'auteur de chaque ligne de code.

Lors de l'épreuve orale, les questions sur le code source ne porteront que sur le code écrit par l'élève. Par contre, les questions sur le cahier des charges porteront sur l'entièreté de ce dernier.

Attention : le niveau d'exigence sera supérieur pour les projets réalisés à deux, tant au niveau de la rédaction du cahier des charges qu'au niveau de la réalisation du programme.

Les différentes grilles d'évaluation sont disponibles à la fin de ce document.

6. Remise finale et nomenclature

Lors de la remise finale vous remettrez une archive compressée nommée **`2606_5TT_Nom(s)_TFA.zip`** comprenant :

1. Le cahier des charges en .pdf

   Nomenclatures : **`2606_5TT_Nom(s)_CahierDesChargesTFA.pdf`**
2. Un dossier nommé **`2606_5TT_Nom(s)_CodeSourceTFA`** contenant le code source ainsi que tous les fichiers nécessaires au bon fonctionnement du programme..

Aucun retard ne sera accepté sauf cas de force majeure. En cas de non remise à la date indiquée, une cote nulle sera attribuée.

7. Modalités de récupération

7.1.1. Conditions d'accès

La récupération est autorisée si :

- L'élève a participé au projet et a remis une première version exploitable (même incomplète).

- Les manquements relèvent d’améliorations possibles (fonctionnalités, dossier, préparation orale) et non d’une absence de travail.

7.1.2. Éléments à représenter

Selon les lacunes constatées, la récupération peut porter sur un ou plusieurs éléments :

- Le code / fonctionnalités attendues.
- Le dossier de conception (mise à jour et corrections).
- La défense orale individuelle.

7.1.3. Exigences minimales

- La version représentée doit corriger explicitement les points signalés lors de la première évaluation.
- Toute amélioration doit rester cohérente avec le cahier des charges et la méthodologie de travail demandée.
- Les livrables sont remis avec la même rigueur de présentation (structure, clarté, orthographe, etc.).

8. Indicateurs de qualité

8.1. La capacité fonctionnelle

« Le programme fait-il ce qu'il est censé faire ? »

La capacité fonctionnelle est la capacité qu'ont les fonctionnalités d'un logiciel à répondre aux exigences et besoins explicites ou implicites des usagers. En font partie la précision, l'interopérabilité, la conformité aux normes et la sécurité.

Exemple : Dans un jeu de Snake , si le serpent mange une pomme, son corps doit s'allonger d'un bloc et le score doit augmenter de 1 point. Si le score ne bouge pas, la capacité fonctionnelle est défaillante.

8.2. La facilité d'utilisation

« Comment utiliser le programme sans devoir lire un manuel de 50 pages ? »

Elle porte sur l'effort nécessaire pour apprendre à manipuler le logiciel. En font partie la facilité de compréhension, d'apprentissage et d'exploitation et la robustesse. Un autre exemple, une utilisation incorrecte n'entraîne pas de dysfonctionnement.

Exemple : Au lieu d'écrire « Appuyez sur la touche 34 pour sauter », utilisez les flèches du clavier ou les touches ZSQD qui sont naturelles pour un joueur. L'ajout d'un bouton « Rejouer » bien visible après une partie perdue améliore aussi cet indicateur.

8.3. La fiabilité

« Le programme plante-t-il ou s'arrête-t-il brusquement ? »

C'est-à-dire la capacité d'un logiciel de rendre des résultats corrects quelles que soient les conditions d'exploitation. En font partie la tolérance aux pannes - la capacité d'un logiciel de fonctionner même en étant handicapé par la panne d'un composant (logiciel ou matériel).

Exemple : Si un élève entre son nom dans le menu et qu'il tape des symboles bizarres (comme @#$%), le jeu ne doit pas s'éteindre ou afficher une page blanche. Il doit rester stable quelles que soient les actions imprévues de l'utilisateur.

8.4. La performance

« Le programme est-t-il une usine à gaz ? »

C'est-à-dire le rapport entre la quantité de ressources utilisées (moyens matériels, temps, personnel), et la quantité de résultats délivrés. En font partie le temps de réponse, le débit et l'extensibilité - capacité à maintenir la performance même en cas d'utilisation intensive.

Exemple : Si un jeu met 10 secondes à charger une image ou si le personnage se déplace avec des saccades (le « lag »), la performance est mauvaise. Un bon code JavaScript doit permettre au jeu de tourner de façon fluide, même sur un vieil ordinateur de l'école.

8.5. La maintenabilité

« Sera-t-il facile de modifier ou d'améliorer le code plus tard ? »

Elle mesure l'effort nécessaire à corriger ou transformer le logiciel. En font partie l'extensibilité, c'est-à-dire le peu d'effort nécessaire pour y ajouter de nouvelles fonctions.

Exemple : Si vous avez utilisé une variable `vitesse = 5` au début de votre code, vous pouvez changer la difficulté du jeu en modifiant un seul chiffre. Si vous avez écrit « 5 » manuellement à 100 endroits différents dans votre code, votre projet n'est pas maintenable.

8.6. La portabilité

« Le programme peut-il fonctionner partout ? »

C'est-à-dire l'aptitude d'un logiciel de fonctionner dans un environnement matériel ou logiciel différent de son environnement initial. En font partie la facilité d'installation et de configuration dans le nouvel environnement.

Exemple **:** un jeu doit fonctionner de la même manière si vous y jouer sur une console de Nintendo, Sony ou Microsoft.

GRILLE D'EVALUATION

IMPLEMENTATION DU PROJET

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le projet selon une nomenclature correcte (MALUS)				
Maitrise technique et processus	Maitrise des techniques de programmation Le projet ne présente aucun bug L'élève a amené des éléments nouveaux (recherches) Le code est documenté Le code est optimisé et permet sa maintenabilité Le code est de qualité				
Produit fini	Le projet représente un ensemble fonctionnel. L'élève a implémenté tous les contenus énoncés dans son dossier de conception. Le projet est original, les règles graphiques sont respectées et cohérentes. Le projet est ergonomique et facile à prendre en main.				
Communication	Le projet ne présente aucune faute d'orthographe.				

Pondération	/100

GRILLE D'EVALUATION

DOSSIER DE CONCEPTION

	Grille d'évaluation				
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Respect des normes, règles et consignes	A respecté les délais (MALUS) A remis le fichier selon une nomenclature correcte (MALUS)				
Produit fini	Le dossier est complet. • Définition du projet • Analyse et conception ◦ Interfaces et cas d'utilisation ◦ Description des données ◦ Diagramme(s) d'action • Implémentation ◦ Structures et énumérations ◦ Prototypes ◦ Fonctions • Rétrospective • Perspectives Le dossier explique les choix de conception de manière complète.				
Communication	Le dossier de conception ne comprend pas de faute d'orthographe. La mise en page, la structure des textes et leur contenu permettent une lecture facile et une compréhension totale du dossier.				

Pondération	/100

GRILLE D'EVALUATION

DEFENSE ORALE INDIVIDUELLE

Grille d'évaluation					
Critères	Indicateurs	Points	Acquisition		
			A	N-A	E-A
Technique et processus	L'élève prouve le développement d'une part équitable du projet. L'élève peut vulgariser avec aisance le fonctionnement général de son code. L'élève maitrise et peut restituer une explication claire d'un algorithme particulier. L'élève maitrise les détails techniques propre aux langages de programmation utilisés. L'élève est capable de proposer une correction de son code et/ou une ou plusieurs fonctionnalité(s) supplémentaire(s).				

Pondération	/100

FEUILLE DE DÉFINITION DU PROJET

Nom des élèves : ……

……

……

Projet : ……

Date :

Signature des élèves

Signature du professeur